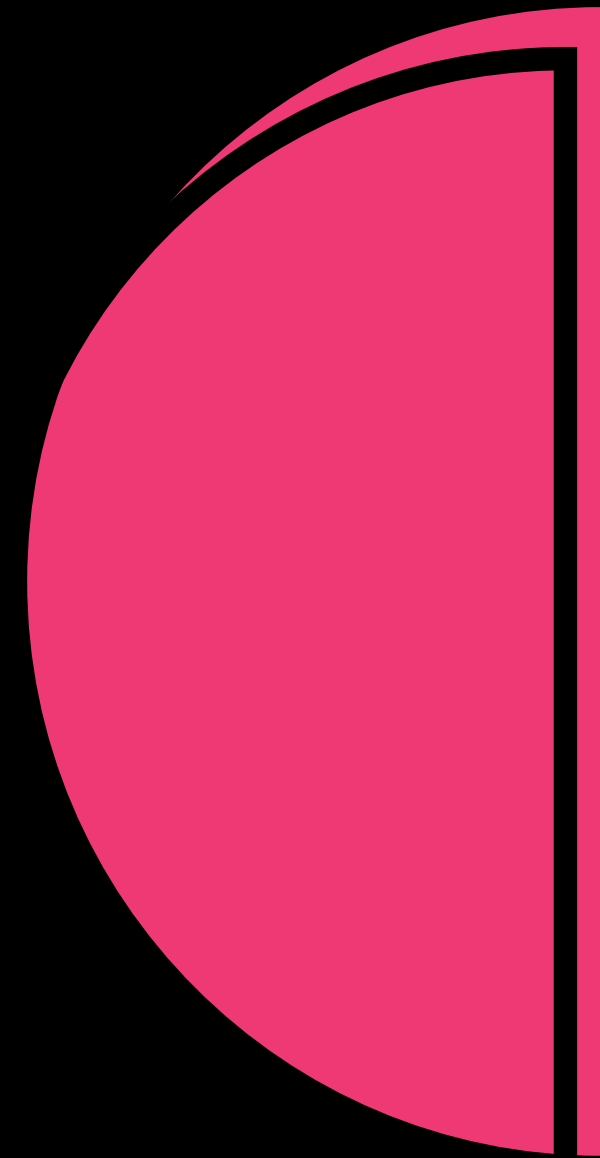




SC2001 PROJECT 3

TEAM 7 - SCSB
Nguyen An Loc
Ooi Jia xi
Panicker Shobith Shreekumar

Start

PROJECT 3

UNBOUNDED KNAPSACK PROBLEM

1. Recursive function $P(c)$

2. Subproblem graph

3. DP Algorithm

4. Python implementation

UNBOUNDED KNAPSACK PROBLEM

Problem Scenario

A knapsack with limited capacity C and several item types of unlimited quantity.

Each with weight (w_i) and profit (p_i).

Goal

Find the **maximum profit** $P(C)$ can be achieved without exceeding C .

Unlike the **0/1** version, each item can be chosen multiple times!

Why Dynamic Programming?

1. **Overlapping subproblems:** same capacities appear repeatedly
2. **Optimal substructure:** best overall solution comes from best smaller ones

Real-life applications

Coin change, rod cutting, resource allocation, production planning, etc

1. RECURSIVE DEFINITION OF $P(C)$

Base case ($C = 0$)

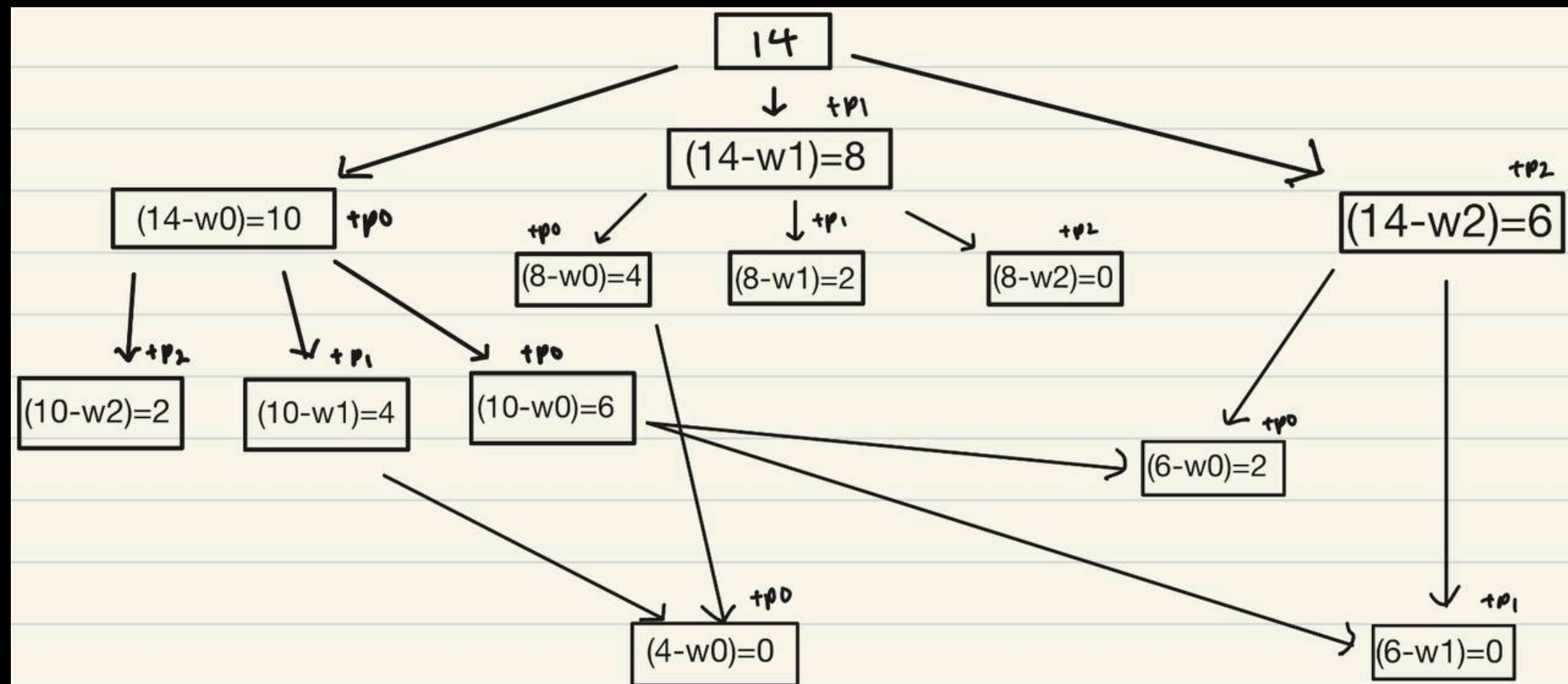
- $P(0) = 0$
- Represents the scenario when the knapsack has a capacity of 0, or has no free space. Hence, no objects can fit into the knapsack, and maximum profit is 0.

Recursive case ($C > 0$)

- $P(C) = \max[P(C - w_i) + p_i]$ if $w_i \leq C$, for $0 \leq i < n$
- For every capacity, we consider the max profit of every object from index 0 to $n-1$.
- If weight $< C$, there is space in the knapsack to store the object allowing us to include it and recursively call how much profit we will get from that.

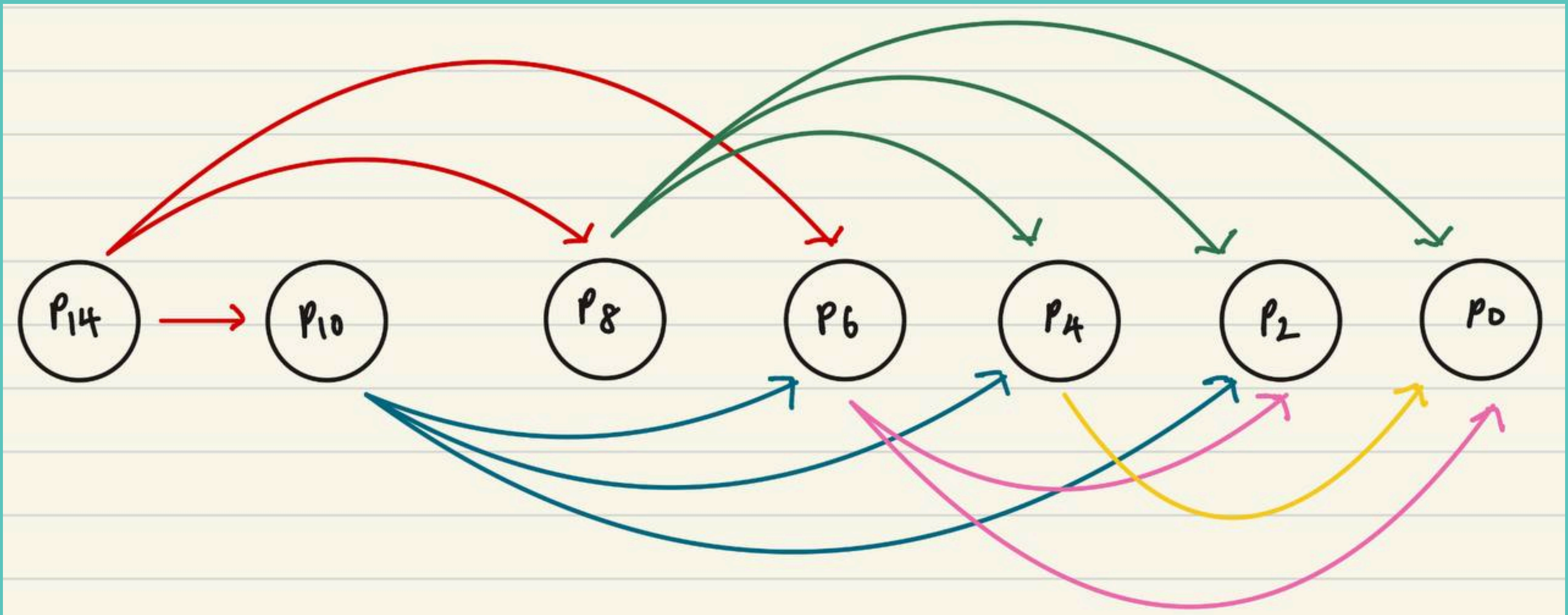
$$P(C) = \begin{cases} 0, & C = 0, \\ \max_{0 \leq i < n, w_i \leq C} (p_i + P(C - w_i)), & C > 0. \end{cases}$$

2. SUBPROBLEM GRAPH



	0	1	2	
w_i	4	6	8	$n=3$
p_i	7	6	9	

2. SUBPROBLEM GRAPH



3. DP ALGORITHM

Pseudo-code

```
def unbounded_knapsack(C, n, p, w):  # C: capacity, n: # of objects  
                                     # p: profit array, w: weight array
```

```
  # Initialize DP array P[0...C]
```

```
  for (c = 0 to C):
```

```
    P[c] = 0
```

Space complexity: $O(C)$

```
  # Iterate through each knapsack capacity
```

```
  for (c = 0 to C):
```

C iterations

```
    # For each capacity, try all object types
```

```
    for (i = 0 to n - 1):
```

n iterations

```
      # Check if object i can fit at this capacity
```

```
      if (w[i] <= c):
```

```
        # Choosing object i: add its profit to the best value
```

```
        # of the remaining capacity (c - w[i])
```

```
        P[c] = max(P[c], P[c - w[i]] + p[i])
```

Time complexity:
 $O(nC)$

```
  return P[C]
```

4. IMPLEMENTATION

```
def knapsack(C, n, p, w):  
    # Initialize 1D DP array  
    P = [0] * (C + 1)      # P[i] = max profit can obtained using knapsack capacity of i  
  
    # Iterate through every knapsack capacity  
    for c in range(1, C + 1):  
        # For each capacity, try all object types  
        for i in range(n):  
            # If current object fit into capacity c  
            if w[i] <= c:  
                # Take the max of two choices:  
                # 1. Skip object i: keep current best value stored in P[c]  
                # 2. Choose object i: Add its profit to P[c - w[i]]  
                P[c] = max(P[c], P[c - w[i]] + p[i])  
  
    return P[C]
```

4. RESULTS

(4a)

	0	1	2
w _i	4	6	8
p _i	7	6	9

```
# (4a): weights = [4, 6, 8], profits = [7, 6, 9]
```

```
C = 14
weights = [4, 6, 8]
profits = [7, 6, 9]
n = len(weights)
```

```
profit_4a = knapsack(C, n, profits, weights)
print("4a: P(14) =", profit_4a)
```

✓ 0.0s

4a: P(14) = 21

(4b)

	0	1	2
w _i	5	6	8
p _i	7	6	9

```
# (4b): weights = [5, 6, 8], profits = [7, 6, 9]
```

```
C = 14
weights = [5, 6, 8]
profits = [7, 6, 9]
n = len(weights)
```

```
profit_4b = knapsack(C, n, profits, weights)
print("4b: P(14) =", profit_4b)
```

✓ 0.0s

4b: P(14) = 16



**Thank you
very much!**